KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
SCHOOL OF ENGINEERING



Data Structures and Algorithms

Project Report: LEBIS — Algorithms for an Equipment Rental System

INSTRUCTED BY: Dr. Anakkapon Saenthon

NAME	STUDENT ID
NITHIT SRISENGSUWANCHAI	68011411
INTUCH SRISAWAT	68011345
ISSAREE AUSAVAPIROM	68011346

DATE OF SUB.: 18/10/2026

Contents

1. Introduction
2. Part 1 — Inserting and Deleting with a Hand-Written Hash Table (Kim)
3. Part 2 — Searching and Counting with a Hand-Written Binary Search (Neo)
4. Part 3 — Sorting and Grouping with a Hand-Written Merge Sort (Kul)
5. Conclusion — Three Algorithms Working Together
6. Appendix — Flowcharts

Introduction

> *Written by the editor (Neo) to set the scene for the three main parts. Every number in this report comes from `LEBIS_Final.ipynb`, the combined notebook of all three members, from one single run.*

The system these algorithms serve

LEBIS is a borrow-and-return system for small equipment rental shops with one to three staff, such as camera rental shops. It runs live at https://lebis-project.vercel.app.

A shop like this asks the same questions every day. This report answers **which data structure each question needs, and why**.

The shop's question	Algorithm group	Author	Part
"Is camera CAM-0012 available?" · receiving stock · removing stock	Inserting · Deleting	Kim — hash table	1
"The lens I rented before" · "How many are free, how many are overdue?" · "What is due before date X?"	Searching · Counting	Neo — binary search	2
"Which rental has been out the longest?" · "What is in each category?"	Sorting · Grouping	Kul — merge sort	3

All three parts implement their algorithms by hand. Python's `dict`, `bisect` and `sorted()` are never used to do the work. They are used only as **baselines**, to show that the hand-written code gives correct results and to explain why the built-ins are faster.

All the code lives in `backend-api/algorithms/` and imports only the Python standard library. The same files therefore run both in the live web application and in Google Colab without changing a line.

The shared data sets

Every part measures on the same data from `make_dataset()` in `datasets.py`, so the numbers of the three parts can be compared directly. The data uses fixed seeds rather than fresh random values, and a fixed reference date `TODAY = 2026-10-01`, so every run produces the same data.

Set	Size	Duplicate names	Out of stock	Overdue	Broken	Built to prove
A	10	0	1	3	1	Every function works in the normal case
B	12	6	5	4	1	Duplicate names — search by code must return the right item, search by name must return every match
C	10	0	5	4	1	Many items out of stock — status and counting at the edges
D	1,000	0	288	275	142	How the Big-O difference shows when the data grows 100 times

Every set has at least 10 records, as the assignment requires. Each record has 8 fields: `code` · `name` · `category` · `total` · `available` · `borrower` · `due_date` · `status`.

How time was measured

All three parts use the same method — `time.perf_counter()`, run 3 times and averaged, excluding the time spent preparing data. Times are in milliseconds.

The measured times vary slightly with the machine on every run, but **the order of which method is faster does not change**. For that reason every number in this report comes from the same single run. Numbers from individual members' notebooks must not be mixed in.

Google Colab

The combined notebook, already run: https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8

> ⚠️ ***Do not press Run all in the shared file.*** *The saved output is the same run as in this report. Running it again changes the times, and the numbers will no longer match the report. To try it yourself, make a copy first (File → Save a copy in Drive).*

Course requirements — where to find them

Every algorithm must include five things. This table shows where each one is for each part.

Requirement	Part 1 (Kim)	Part 2 (Neo)	Part 3 (Kul)
Big-O for time and memory, with an explanation	1.5	2.6	3.5
Flowchart	1.8 · Appendix	2.9 · Appendix	3.10 · Appendix
Real results on sets A–D	1.7	2.8	3.7
Time on set D compared with what Python provides	1.7 — `dict`	2.8 — scanning the whole list	3.7 — `sorted()`
Colab link	1.9	2.10	3.11

All seven flowcharts are collected in the Appendix at the end, in figure order.

Part 1 — Inserting and Deleting with a Hand-Written Hash Table (Kim)

> *English translation of `kim-hashtable.md`, which was written by Kim. The content, analysis and numbers are Kim's; only the language has changed. Every number comes from `LEBIS_Final.ipynb` (the combined notebook of all three members), committed in the repository — the same file linked from Colab.*

1.1 The process chosen

A photography equipment rental shop with one to three staff repeats the same questions every day.

- A customer asks *"Is camera CAM-0012 available?"* — an item has to be found by its code.
- New stock arrives — an item has to be added, and codes must never repeat.
- Items are rented out and returned — the available count has to change.
- An item breaks beyond repair — it has to be removed from the system.

If items are kept in a plain list, every question means scanning one item at a time until a match is found, and the more stock there is, the slower it gets. This part therefore chooses the **Inserting** and **Deleting** algorithm groups (including lookup by code), and implements the hash table entirely by hand rather than wrapping Python's `dict`.

1.2 Data structure

Each item is stored as a record with 8 fields, shared by the whole team.

`code` · `name` · `category` · `total` · `available` · `borrower` · `due_date` · `status`

The system keeps items **in two places side by side**.

```
items = [ {CAM-0001 ...}, {LEN-0002 ...}, ... ]   records, in order
index = HashTable: "CAM-0001" -> 0 · "LEN-0002" -> 1   code -> position
```

Stored as	Lookup by code	Sorting or walking in order
List only	O(n)	Yes
Hash table only	O(1) average	No — the order is the order of the buckets
List + hash index	**O(1) average**	**Yes**

Both are needed because other parts of the system need order: Kul's merge sort sorts rentals by due date, and Neo's binary search finds what is due before a given date. A hash table cannot answer range or ordered questions at all, so the list cannot be dropped. The price is the memory for one extra table, and the need to keep the two in step at all times.

1.3 The hash table

Hash function

A **polynomial rolling hash** is used.

```
h = 0
for every character c in the code:   h = (h × 31 + ord(c)) mod 1,000,000,007
bucket = h mod number_of_buckets
```

- **Character position matters** — `CAM-01` and `CAM-10` land in different buckets. Simply adding the character codes would give these two the same value, and this is the code format shops really use.
- **Base 31** is prime — small enough to multiply quickly, large enough to spread the values.
- **Taking the result modulo a large prime** stops the number growing without limit and slowing down the calculation.

Collisions and chaining

A collision is when two different codes hash to the same bucket. It cannot be avoided, because codes are unlimited but buckets are not. **Chaining** is used — one bucket holds several pairs linked as a chain. A lookup goes to that bucket and scans the short chain.

Resizing

When the **load factor** (number of items ÷ number of buckets) exceeds **0.7**, the number of buckets doubles and every item is placed again (rehash). The load factor is the average chain length, so it is the number that says whether the table is still fast.

- Too low — the table is sparse, wastes memory, and resizes often.
- Too high — saves space, but chains get long and lookups slow down.

A resize costs O(n) each time, but it only happens when the data has doubled. Averaged over every insert, the cost is therefore still O(1) (amortized O(1)).

1.4 Adding, renting, returning and removing

Every function validates first and changes data only if validation passes. If the list were changed and the index update then failed, the two would be out of step forever.

Function	Rejected when
`add_item`	Duplicate code · invalid quantity
`borrow_item`	Unknown code · item broken · no unit available · no renter name
`return_item`	Unknown code · item not currently rented
`remove_item`	Unknown code · units still with customers

Swap-and-pop removal — removing from the middle of a list directly is O(n), because every later item has to shift, and the positions the index remembers for them all become wrong. Instead, the last item is

moved into the removed position, only that moved item's index entry is updated, and the last slot is dropped. Every step is O(1). The price is that the order of the list changes, which is acceptable because any task that needs order sorts the data itself.

1.5 Complexity (Big-O)

Operation	Average	Worst	Worst case happens when
Lookup by code	O(1)	O(n)	Every code lands in the same bucket
Insert	O(1) amortized	O(n)	The insert triggers a resize, or a chain is long
Delete	O(1)	O(n)	A chain is long
Resize	O(n) per resize	O(n)	—

Why O(1) is the average and not the worst case — if every code hashed to the same bucket, the table would be a single chain of length n, and a lookup would become a one-by-one scan, just like a list. This can happen when the hash function spreads values poorly, when the table never resizes, or when someone deliberately chooses codes that all collide. Section 4 of the notebook shows a table that is not allowed to resize, for comparison: the average chain is 15 items long and the longest is 17 (a normally resizing table's longest chain is only 3). The more data is added, the longer the chains grow with n, which is O(n) in practice. In short, O(1) is not free: it comes from a well-spread hash function together with resizing; without either one, it degrades to O(n).

1.6 Test data sets

The same `make_dataset()` by Neo is used across the team. Every set has at least 10 records, as the assignment requires.

Set	Size	Characteristics	Used to prove
A	10	Clean data	Every function works
B	12	Duplicate names, unique codes	Lookup by code returns the right item
C	10	Many items out of stock	Status and counting
D	1,000	Large set	The Big-O difference as data grows

1.7 Results

Three methods are measured on the same data — a list scan, the hand-written hash table, and Python's `dict` (the baseline). Each cell is run 3 times and averaged with `time.perf_counter()`; time spent preparing data is excluded. Times are in milliseconds · "Speed-up" = list scan time ÷ hand-written hash table time.

Table H1 — Look up every code once

Set	List scan	HashTable	dict	Speed-up
A (10)	0.004	0.015	0.001	0.3×
B (12)	0.005	0.017	0.001	0.3×
C (10)	0.004	0.014	0.001	0.3×
D (1,000)	21.753	1.530	0.053	**14.2×**

Table H2 — Add every record one at a time (every method checks for a duplicate code first)

Set	List scan	HashTable	dict	Speed-up
A (10)	0.004	0.042	0.002	0.1×
B (12)	0.005	0.066	0.002	0.1×
C (10)	0.004	0.041	0.002	0.1×
D (1,000)	20.625	5.902	0.112	**3.5×**

Table H3 — Delete every code once

Set	List scan	HashTable	dict	Speed-up
A (10)	0.006	0.019	0.002	0.3×
B (12)	0.005	0.020	0.001	0.3×
C (10)	0.005	0.016	0.001	0.3×
D (1,000)	15.152	1.719	0.061	**8.8×**

Table H4 — Hash table state after inserting the whole set

Set	Items	Buckets	Collisions	Resizes	Longest chain	Load factor
A	10	16	2	1	2	0.625
B	12	32	5	2	3	0.375
C	10	16	2	1	2	0.625
D	1,000	2,048	374	8	3	0.488

Reading the results

1. **On the small sets (A–C) the list scan is faster** — scanning 10 items costs less than computing a hash each time. Big-O describes behaviour as n grows; it does not say who wins at small n. Measuring only the small sets would lead to the wrong conclusion.
2. **On set D the hash table wins all three tasks** — the data grows 100 times from set A. Lookup time with a list scan grows about 5,438 times (n lookups, each scanning half the list on average = $n^2/2$), while the hash table grows about 102 times, in proportion to n only.
3. **Inserting wins by the smallest margin (3.5×)** because the hash table spends time resizing 8 times while inserting 1,000 records. This cost is spread across the whole set (amortized), so it still grows with n, not n^2.

4. **Many collisions, but short chains** — set D has 374 collisions, yet the longest chain is only 3, because the table resizes every time the load factor exceeds 0.7. This is evidence that collisions are not a problem as long as chains are not allowed to grow long.
5. **Python's `dict` is faster than ours** — about 28–29 times for lookup and delete, and about 53 times for insert (`dict` resizes itself in C, while ours rehashes one item at a time in Python). Both are O(1), however; they differ by a constant factor and grow at the same rate.

The measured times vary with the machine on every run, but the order of which method is faster does not change.

1.8 Flowcharts

In `docs/data-course/flowcharts/`, as both draw.io and PDF files · the PDFs are also in the team folder: https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

1. `flow_kim_1_overview` — system overview: every command goes through the hash index first
2. `flow_kim_2_insert` — adding an item, including the paths for a collision and for resizing
3. `flow_kim_3_delete_return` — swap-and-pop removal, and returning an item

1.9 Related files

File	What it is
`backend-api/algorithms/hash_table.py`	All the code; standard library only
`docs/data-course/LEBIS_Final.ipynb`	The combined, already-run notebook of all three members — the numbers in this report come from this file · **Colab:** https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/LEBIS_Kim_HashTable.ipynb`	Kim's own notebook, with more detailed explanations (its numbers are from a different run; do not mix them)
`scripts/data-course-hashtable.py`	Script that measures sets A–D locally
`scripts/build-notebooks.py`	Builds the notebooks from the real files in the repository

Part 2 — Searching and Counting with a Hand-Written Binary Search (Neo)

> *English version of `neo-search-count.md`. Every number comes from `LEBIS_Final.ipynb` (the combined notebook of all three members), committed in the repository — the same file linked from Colab.*

2.1 The process chosen

The previous part solved "where is the item with this code?" with a hash table, which is the fastest answer when the exact code is known. The questions a shop really asks go further than that.

- A customer cannot remember the code and only gives a name: *"the lens I rented last time"* — items must be found by part of their name, and names can repeat.
- The owner opens the system in the morning — they need to know **how many units are free, how many are with customers, and how many rentals are overdue**.
- Before closing — they need to know *"what is due back before date X?"*, which is a **range** question, not a single value.

A hash table cannot answer the last two at all, because a hash works out a position only from an exact key. This part therefore chooses the **Searching** and **Counting** algorithm groups, with binary search written entirely by hand. Python's `bisect` is not used, except as a baseline for comparison.

2.2 Data structure and the precondition for searching

The whole team uses the same 8-field record from `make_dataset()`.

`code` · `name` · `category` · `total` · `available` · `borrower` · `due_date` · `status`

Binary search needs one thing a plain list does not have: **the data must already be sorted**.

> 🔴 *If the list is not sorted by `code`, the halving cuts off the wrong side and **returns `None` even though the item is there**. No error reports this mistake; someone would have to notice it by eye. That is why `sort_by_code()` is provided to be called first, and why it returns a new list instead of changing the original.*

In the live system, the list sorted by `due_date` comes straight from **Kul's merge sort** (`sort_by_due_date()`). The two pieces of work connect, so we agreed that every function returns a new list and never changes a list someone else holds.

2.3 Three search paths

The path is chosen by **the kind of question**, not by which method is fastest.

Path	Function	Used when	Time
Linear	`linear_search`	Not sorted, no hash table; used as the baseline	O(n)
Hash	`hash_lookup`	The exact code is known and Kim's table is available	O(1) average
Binary	**`binary_search`**	**Exact code on a sorted list — the hand-written one**	**O(log n)**
Partial name	`search_by_name`	The customer only remembers the name	O(n)

How binary search works — look at the middle item first. If it matches, stop. If the target is smaller than the middle, it must be in the left half, so the whole right half is discarded, and vice versa. **Every comparison halves what is left to examine.** 1,000 items therefore need only about 10 comparisons, because $2^{10} = 1{,}024$.

It is written as a loop rather than recursion, because a loop uses no extra stack space, **so memory is O(1), not O(log n)**. It uses `middle = low + (high - low) // 2` instead of `(low + high) // 2`, the standard way to avoid overflow in languages where `int` has a fixed size (Python does not overflow, but the correct form is kept).

`search_by_name` must return every match, not just the first, because names really do repeat in a shop. Returning the first match, as `linear_search` does, would give the user the wrong item without them knowing — renting out the wrong unit, while the other item silently disappears from the system. Set B was built specifically to prove this.

2.4 Range questions — where a hash table cannot help

`find_due_before(sorted_by_due, limit_date)` answers *"what is due back before date X?"*

It uses halving to find the leftmost position where `due_date >= limit_date` in O(log n), then takes every item before it in one block, for a total of **O(log n + k)**, where k is the number of answers — which have to be copied out whatever method is used.

Note that `high` starts at `len()` rather than `len() - 1`, because the answer may be "past the end", meaning that no item reaches the limit.

A hash table cannot do this. A hash only knows which bucket an exact key is in; it has no idea of "before" or "after". A range question would have to scan the whole table, which is O(n) — **this is why the project still needs binary search even though it already has Kim's hash table**.

2.5 Counting

Three functions share exactly the same structure: one pass over the data, differing only in the condition and in what is added.

Function	Counts	Easy to get wrong
`count_available`	**Units** free on the shelf	Broken units do not count — they may sit on the shelf, but cannot be rented
`count_by_status`	**Records** per status	Always returns all 3 keys, even when a value is 0
`count_overdue`	Rentals past their due date	Returned items have no `due_date` and must be skipped first, otherwise `TypeError`

All three are O(n) and there is no shortcut — counting differs from searching in that there is no "right one" to stop at. Every item has to be examined before the number is known. Binary search cannot help, because halving needs a single target that can be compared as greater or smaller, and a hash table only answers "where is this one", not "how many meet the condition".

`count_by_status` produces all three numbers in a single pass rather than three separate loops — the same result, reading the data three times less.

Dates can be compared as plain strings, because `YYYY-MM-DD` runs from the largest unit to the smallest. `"2026-09-26" < "2026-10-01"` is true both as text and as dates — stored as `DD/MM/YYYY`, this would stop being true at once.

2.6 Complexity (Big-O)

Function	Time	Memory	Worst case happens when
`linear_search`	O(n)	O(1)	The item is not there at all, so every item must be checked
`binary_search`	**O(log n)**	**O(1)**	The item is at the very edge, using all $\log_2(n)$ rounds
`hash_lookup`	O(1) average	O(n) for the table	Every code lands in the same bucket, making it O(n)
`search_by_name`	O(n)	O(k) for the results	Every record contains the search term
`sort_by_code`	O(n log n)	O(n)	—
`find_due_before`	**O(log n + k)**	O(k)	Every record matches (k = n)
`count_available` · `count_by_status` · `count_overdue`	O(n)	O(1)	Every case is the same; there is no shortcut

Why binary search uses O(1) memory even though it halves repeatedly — because it is a loop, not recursion. It uses only three variables, `low`, `high` and `middle`, however much data there is. A recursive version would use one stack frame per level, making memory O(log n).

2.7 Test data sets

The team shares the same `make_dataset()`, written in this part (`datasets.py`), with the reference date fixed at `TODAY = 2026-10-01`.

Set	Size	Duplicate names	Out of stock	Overdue	Broken	Used to prove
A	10	0	1	3	1	Every function works in the basic case
B	12	6	5	4	1	`search_by_name` must return every match
C	10	0	5	4	1	Counting when many items are out of stock
D	1,000	0	288	275	142	The Big-O difference as data grows

The data uses fixed seeds rather than fresh random values, **so the numbers in this report repeat on every run**. A broken item is placed at every 7th position so that each small set has at least one; otherwise the "skip broken items" condition in `count_available` would never be tested.

2.8 Results

Each cell is run 3 times and averaged with `time.perf_counter()`, the same method as Parts 1 and 3. Time spent preparing data is excluded. Times are in milliseconds · "Speed-up" = linear time ÷ binary time.

Table S1 — Look up every code once

Set	Size	Linear	Binary	Hash	Speed-up
A	10	0.004	0.006	0.016	0.6×
B	12	0.004	0.007	0.018	0.6×
C	10	0.003	0.006	0.015	0.6×
D	1,000	21.812	1.759	1.580	**12.4×**

The Python baseline in this table — item 4 of `plan.md` assigns a different baseline to each member's work: `dict` for the hash table · `sorted()` for sorting · **scanning the whole list for searching**. The linear column in Table S1 is that baseline — a plain Python loop over the items, with no data structure to help. The hash column is Kim's `HashTable` from Part 1, included to show where the third method stands on the same data.

Neo's own notebook (`LEBIS_Neo_SearchCount.ipynb`, section 6) also measures against Python's `bisect`. The times are of the same order; `bisect` is slightly faster because it is written in C, not because its algorithm is better. **Those numbers are not included in this table**, because they come from a different run, and mixed numbers cannot be compared.

Table S2 — Counting results

Set	Free on shelf (units)	ok	out	broken	Overdue
A	24	9	0	1	3
B	14	7	4	1	4
C	10	5	4	1	4
D	1,309	616	242	142	275

Table S3 — Searching by a duplicated name (set B)

Searching for `"เลนส์ รุ่นที่ 1"` ("Lens model 1") returns **4 records** — `LEN-0001` (0/2, out) · `LEN-0002` (3/5, ok) · `LEN-0011` (4/5, ok) · `LEN-0012` (0/5, out)

Of these, 2 names match exactly. The others contain the search term inside their names — *"model 11"* contains *"model 1"*. **This is the right behaviour for the search box on the website**, because users can type part of a name and still find it.

Table S4 — A range question (set D)

"Which items are due back before 2026-10-01?" — binary search finds **275 records** · a one-by-one O(n) count finds **275 records → identical**

Reading the results

1. **On the small sets (A–C), linear is faster than binary** (0.004 vs 0.006 ms) — there is too little data for halving to repay the cost of computing positions. **O(log n) does not mean always faster; it means growing more slowly.** Concluding from the small sets that binary search is worse would be the wrong conclusion.
2. **On set D, binary is 12.4 times faster than linear** (21.812 vs 1.759 ms), and hash is 13.8 times faster than linear (1.580 ms).
3. **The growth rates match the theory almost exactly** — the data grows 100 times from A to D, and this measurement looks up every code, that is, n lookups, so we expect:
   - binary: n × O(log n) → should grow 100 × 3 = **300 times** · measured 1.759 ÷ 0.006 = **293 times**
   - hash: n × O(1) → should grow **100 times** · measured 1.580 ÷ 0.016 = **99 times**

   Both match what Big-O predicts, closely enough that it is not a coincidence.
4. **Linear grows less than the theory says** — n × O(n) should grow 10,000 times, but it measured 21.812 ÷ 0.004 ≈ 5,453 times. This is not because the theory is wrong: at n = 10, most of the time goes on fixed costs (calling the function, setting up the loop) rather than on actual comparisons, so the divisor is too large, **which makes the ratio look smaller**. This is the same reason as point 1, seen from another angle.
5. **Hash and binary are very close on set D** (1.580 vs 1.759 ms), even though their Big-O classes differ, because $\log_2(1{,}000) \approx 10$ is still a small number. The difference would become clear only at much larger n. **But speed is not the reason binary search was chosen** — the reason is Table S4, which a hash table cannot answer.
6. **Counting has no speed comparison table**, because there is no other method to compare with: every method has to examine every item. Table S2 therefore reports the correctness of the numbers, not their times.

The measured times vary with the machine on every run, but the order of which method is faster does not change.

2.9 Flowcharts

In `docs/data-course/flowcharts/`, as both draw.io and PDF files · the PDFs are also in the team folder: https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

1. `flow_neo_4_search` — searching, showing all 3 paths, with the binary search loop drawn step by step
2. `flow_neo_5_count` — the 3 counting functions side by side, with the reason all of them are O(n)

2.10 Related files

File	What it is
`backend-api/algorithms/search.py`	All the searching and counting code; standard library only
`backend-api/algorithms/datasets.py`	Data sets A–D shared by the whole team
`docs/data-course/LEBIS_Final.ipynb`	The combined, already-run notebook of all three members — the numbers in this report come from this file · **Colab:** https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/LEBIS_Neo_SearchCount.ipynb`	Neo's own notebook, with more detail (its numbers are from a different run; do not mix them)
`scripts/data-course-benchmark.py`	Script that measures sets A–D locally, combining all three modules

2.11 This code runs in the live system, not just for the assignment

The Workspace page of the website shows three numbers, and two of them call functions from this part directly, through `services/inventory.py`.

- `available_units` ← `count_available()`
- `overdue_borrows` ← `count_overdue()`

SQL `COUNT`/`SUM`, which would be faster, is deliberately not used. Counting a second time in SQL would mean defining "free" and "overdue" in two places, **and one day the two would disagree without anyone noticing**.

Before switching, we proved that every number stayed the same, on normal data and on edge cases (a broken item still holding available units · a rental due exactly today · a rental that became overdue yesterday).

`find_due_before()` runs every morning at 08:00 in the summary of items due back that the system sends the shop owner on LINE. The system first sorts the rentals with Kul's merge sort, then uses binary search to take out the overdue rentals, those due today and those due tomorrow in one block — the same sequence described in the Conclusion of this report.

What is not in the website is `binary_search()` for lookup by code. The website uses a database index to find an item by its code instead; `binary_search()` is measured in the notebook only, for comparison with Kim's hash table.

Part 3 — Sorting and Grouping with a Hand-Written Merge Sort (Kul)

> *English translation of `kul-sort-group.md`, Kul's section. The content, analysis and numbers are Kul's; only the language has changed. Every timing number comes from `LEBIS_Final.ipynb` (the combined notebook of all three members), sections 4 and 5, committed in the repository — the same file linked from Colab.*

3.1 The process chosen

A rental shop with one to three staff asks another kind of question that Parts 1 and 2 do not answer: questions about the stock **as a whole**.

- *"Which rental has been out the longest?"* — rentals have to be sorted by due date.
- *"Which item is running low and needs restocking first?"* — items have to be sorted by how many are available.
- *"Which cameras does the shop have? Which lenses?"* — items have to be separated by category.
- *"What is broken right now?"* — items have to be separated by status.

This part therefore chooses the **Sorting** and **Grouping** algorithm groups, split by the kind of field.

Field	Kind of value	What is done	Why
`due_date`	A date — one comes before another	**Sort**	"Out the longest" is the smallest value, so order matters
`available`	A whole number — more or fewer	**Sort**	"Running low" is the smallest value, so order matters
`category`	A label, such as camera · lens	**Group**	A camera does not come before or after a lens; there is no order to sort by, the items only need to be kept together
`status`	A label: `ok` · `out` · `broken`	**Group**	The same — the question is what each status contains, not which status comes first

The hand-written algorithm in this part is **merge sort**; Python's `sorted()` and `.sort()` are not used to do the work. **Bubble sort** is written as a comparison, to show how far apart O(n²) and O(n log n) are as the data grows. `sorted()` is used only as a comparison (baseline).

3.2 Data structure

The input is the same 8-field record used by the whole team.

`code` · `name` · `category` · `total` · `available` · `borrower` · `due_date` · `status`

Function	Takes	Returns
`bubble_sort` · `merge_sort`	A list of records and a `key` function, as `sorted()` does	**A new list**, sorted
`sort_by_due_date` · `sort_by_available`	A list of records	A new list, sorted
`group_by_type` · `group_by_status`	A list of records	A `dict` — the keys are category or status names, the values are lists of the records in that group

A new list is always returned and the original is never changed, because someone else may hold the same list (the same as `sort_by_code` in Neo's `search.py`). This also allows the three methods to be timed on one list: the first method does not leave the data sorted and give the next one an advantage.

The file imports only the Python standard library and no team member's file; all data comes in through parameters. The same file therefore runs both in the live web application and in Google Colab.

3.3 Sorting

Merge sort — split in half, then merge

```
merge_sort(items, key):
    if len(items) ≤ 1:  return list(items)       ← a single item is always sorted
(base case)
    middle = len(items) // 2
    left  = merge_sort(items[:middle], key)      ← call itself on the left half
    right = merge_sort(items[middle:], key)      ← call itself on the right half
    return _merge(left, right, key)
```

`_merge` combines two sorted lists into one list that is still sorted. It compares the front item of each side and takes the smaller one first. When one side runs out, the rest of the other side is already sorted, so it is appended as it is.

```
i = j = 0  · merged = []
while i < len(left) and j < len(right):
    if key(left[i]) <= key(right[j]):   take left[i],  then i += 1
    otherwise:                          take right[j], then j += 1
append left[i:] and right[j:] to merged
```

It is written as recursion because repeated halving fits that form naturally. The depth is about $\log_2(n)$ levels — about 10 levels for 1,000 records, so there is no risk of stack overflow.

Stable sort — why <= and not <

A stable sort is one where **items with equal values stay in their original order**.

In `_merge`, the left side always holds the items that came first in the original data. When the values are equal, <= takes the left item first, so the original order is kept. With <, the right item would be taken first, and equal items would swap places for no reason. This single character is the one thing that makes this merge sort stable.

The effect in the real system: rentals due on the same day stay in the order the database returned them (by id), instead of swapping around every time the page is opened. `verify()` in the file checks this with equal values and compares the result with `sorted()`, which is stable.

`bubble_sort` is stable as well, because it swaps only when the first item is **greater** than the next (>); equal values are not swapped.

Bubble sort — the comparison

```
result = list(items)                        ← copy the list first; the original is not
changed
for each round round_index = 0 … n-1:
    swapped = False
    for i = 0 … n - round_index - 2:
        if key(result[i]) > key(result[i+1]):  swap, then swapped = True
    if no swap happened in this round:  stop    ← already sorted; no need to continue
```

In each round the largest remaining value moves to the end, so the next round is one item shorter. Stopping when no swap happens means data that is already sorted finishes in one round.

Functions the system calls

Function	What it does
`sort_by_due_date(items)`	Removes records whose `due_date` is `None`, then calls `merge_sort` with `key = due_date`, smallest first, so the rental out the longest (the oldest date) is at the top
`sort_by_available(items)`	The same `merge_sort`, with only `key = available` changed; the item with the fewest left is at the top

Why `sort_by_due_date` has to remove `None`: an item nobody has rented has no due date to sort by in the first place, and Python cannot compare `None` with text (it raises `TypeError` at once). The result therefore contains only rentals that have a due date — **685 of the 1,000 records** in set D.

`due_date` is stored as text in the form `YYYY-MM-DD`, which sorts as text in the same order as the real dates (year first, then month, then day), so it can be compared with <= directly, without converting it to a date first.

3.4 Grouping

Both functions pass over the data **once** and drop each record into a bucket by its key.

```
group_by_type(items):
    groups = {}
    for each item:
        groups.setdefault(category, []).append(item)
    return groups

group_by_status(items):
    groups = {"ok": [], "out": [], "broken": []}
    for each item:
        if status is in groups:  groups[status].append(item)
    return groups
```

	group_by_type	**group_by_status**
Keys known in advance?	No — a new key is created as soon as a new category appears (`setdefault`)	Yes — there are only 3 statuses, all created up front
Keys returned	Only the categories actually found in the data	Always all 3 keys, even if a group is empty
Unknown values	—	Skipped; nothing breaks

`group_by_status` always returns all 3 keys, for the same reason as Neo's `count_by_status`: code that uses the result does not have to guard every access with `.get()`.

It differs from Neo's `count_by_status` in that the latter keeps only **counts**, while `group_by_status` keeps **whole records**. It is used when the question is "what is there", not just "how many".

Why one pass, not a search per category — searching only for cameras first, then searching again for lenses, reads the whole data again once per category. The team's data has 5 categories, so it would be read 5 times. The bucket method reads it once, however many categories there are.

3.5 Complexity (Big-O)

n = number of records · m = number of rentals with a `due_date` · k = number of categories

Function	Time, best	Time, average	Time, worst	Extra memory
`bubble_sort`	O(n)	$O(n^2)$	$O(n^2)$	**O(n)** — copies the list before sorting
`merge_sort`	O(n log n)	O(n log n)	O(n log n)	O(n) + stack O(log n)
`sort_by_due_date`	O(n + m log m)	O(n + m log m)	O(n + m log m)	O(m)
`sort_by_available`	O(n log n)	O(n log n)	O(n log n)	O(n)
`group_by_type` · `group_by_status`	O(n)	O(n)	O(n)	O(n)
Searching one category at a time (comparison)	O(n × k)	O(n × k)	O(n × k)	O(n)

Merge sort — O(n log n) in every case

- **How many levels** — halving every time until one item is left, n → n/2 → n/4 → … → 1, takes $\log_2(n)$ steps.
- **Work per level** — at one level, all the `_merge` calls together take each of the n items exactly once = O(n).
- **Total** — O(n) per level × $\log_2(n)$ levels = **O(n log n)**.
- **Why there is no best or worst case** — `merge_sort` always splits at `len(items) // 2` and never checks whether the data is already sorted. Sorted data is still split and merged at every level, so the number of levels is the same in every case.
- **Memory O(n)** — `_merge` creates a new `merged` list, and `items[:middle]` · `items[middle:]` copy the data into new lists. At any moment, the lists still in use belong to the levels nested at that point, of sizes n/2 + n/4 + …, which add up to only a few times n, so the memory is O(n). The recursion is nested $\log_2(n)$ levels deep, a stack of O(log n).

Bubble sort — $O(n^2)$ average and worst, O(n) best

- **Worst** — round 1 compares n−1 pairs, round 2 compares n−2 pairs … in total (n−1) + (n−2) + … + 1 = n(n−1)/2 = **$O(n^2)$**.
- **Best** — when the data is already sorted, the first round compares n−1 pairs with no swap at all, the stop condition triggers, and it ends = **O(n)**. Without this stop condition, bubble sort would always run every round, $O(n^2)$ even on sorted data.
- **Memory O(n), not O(1)** — textbook bubble sort swaps in place and needs O(1) extra memory, but the version in this code starts with `result = list(items)`, copying the whole list first so the original is not changed, so it uses **O(n)** extra memory. The swaps after that happen in place on the copy.

sort_by_due_date — O(n + m log m)

Removing `None` with a list comprehension reads all n records = O(n) and produces a new list of m rentals = O(m) memory. Calling `merge_sort` on the m rentals = O(m log m), giving **O(n + m log m)** in total. When m is close to n this is O(n log n) — set D has m = 685 out of n = 1,000.

Grouping — O(n)

- **Time** — one pass of `for item in items`, with constant work per record: finding the bucket in a `dict` (O(1) on average) and `append` (O(1) amortized). n records therefore take **O(n)**, whatever the number of categories.
- **Memory O(n)** — every record goes into some bucket and none is discarded (except unknown statuses in `group_by_status`). The buckets hold **references** to the original records, not copies — unlike Neo's counting, which uses O(1).
- **Searching one category at a time** — reads all n records k times = **O(n × k)**. With the team's k = 5, that is 5 times the reading of the bucket method.

3.6 Test data sets

The same `make_dataset()` by Neo is used across the team. Every set has at least 10 records, as the assignment requires.

Set	Size	Characteristics	Used to prove
A	10	Clean data	Every function works
B	12	Duplicate names, unique codes	Duplicate names make no difference, because names are not used as a key for sorting or grouping
C	10	5 of 10 items out of stock	Many records share `available = 0` — the case with many equal values
D	1,000	Large set · 685 records with a `due_date`	The Big-O difference as data grows

3.7 Results

Three methods are measured on the same data — bubble sort, the hand-written merge sort, and Python's `sorted()` (the baseline). **Each method sorts the whole set once by the `available` field** (`report_on_datasets` uses `key = item["available"]`), not by `due_date`. Each cell is run 3 times and averaged with `time.perf_counter()`; time spent preparing data is excluded. Times are in milliseconds · "Speed-up" = bubble sort time ÷ merge sort time.

Table M1 — Sort the whole set once by `available`

Set	Bubble sort	Merge sort	`sorted()`	Speed-up
A (10)	0.011	0.013	0.001	0.9×
B (12)	0.014	0.015	0.001	0.9×
C (10)	0.008	0.013	0.001	0.6×
D (1,000)	90.516	2.619	0.124	**34.6×**

`sort_by_due_date`, which the system really uses, is called only on rentals that have a `due_date` (685 in set D). The notebook does not time this function separately, so the numbers in Table M1 are the time of `merge_sort` on the whole set by `available`.

Table G1 — Grouping by category (`group_by_type`): number of records in each group

The category values are stored in Thai in `datasets.py`; the column headings give their English meaning.

Set	Camera	Lens	Tripod	Light	Microphone	Total
A (10)	2	2	2	2	2	10
B (12)	2	4	2	2	2	12
C (10)	2	2	2	2	2	10
D (1,000)	200	200	200	200	200	1,000

Table G2 — Grouping by status (`group_by_status`): number of records in each group

Set	ok	out	broken	Total
A (10)	9	0	1	10
B (12)	7	4	1	12
C (10)	5	4	1	10
D (1,000)	616	242	142	1,000

> *Sets A–C come from running `group_by_type` / `group_by_status` on `make_dataset()` in the repository (fixed seeds, the same result on every run) · set D matches section 4 of the notebook.*

Reading the results

1. **On the small sets (A–C) bubble sort is faster than or about the same as merge sort** (0.9× · 0.9× · 0.6×). Merge sort has a fixed cost at every level — slicing creates new lists, it calls itself again, and it creates a new `merged` list. At 10 records the real work is smaller than these costs, while bubble sort copies the list once and then swaps in place. Big-O describes behaviour as n grows; it does not say who wins at small n. Measuring only the small sets would lead to the wrong conclusion.
2. **On set D merge sort is 34.6 times faster than bubble sort** (90.516 against 2.619 ms). The data grows 100 times from set A. Bubble sort's time grows from 0.011 to 90.516 ms — several thousand times; by the n^2 theory, data that grows 100 times should take about 10,000 times as long. Merge sort grows from 0.013 to 2.619 ms — in the hundreds, below the 300 times that the n log n theory gives. (The small sets' times are in microseconds, so these ratios are rough and show only the trend.)
3. **`sorted()` is the fastest on every set** — on set D it takes 0.124 ms against 2.619 ms for the hand-written merge sort. `sorted()` is Timsort written in C, while our merge sort runs every step as a Python loop and calls `key()` again on every comparison, whereas `sorted()` computes each key once per record. Both are O(n log n), however; the difference is a **constant factor, not Big-O** — they grow at the same rate as the data grows.
4. **Grouping loses no records and adds none** — set D splits into 5 categories of 200 each, 1,000 in total, and by status into 616 + 242 + 142 = 1,000, every record accounted for. The status counts

match Neo's `count_by_status` from the same run (Table 2 in the notebook) in all three columns. All of this comes from a single pass over the data; searching one category at a time would read the data 5 times for the same result.

The measured times vary with the machine on every run, but the order of which method is faster does not change.

3.8 Working with Neo's and Kim's parts

Section 5 of the notebook answers the question the shop asks every morning: **"Which rentals are overdue, and what is each item?"** It needs all three parts in sequence. Each step passes a plain list to the next, and nobody imports anyone else's file.

Step	Who	What it does	Time
1	Kul — `sort_by_due_date`	Sorts the rentals by due date	O(n + m log m)
2	Neo — `find_due_before`	Cuts out every rental due before today with binary search	O(log m + k)
3	Kim — hash table	Looks up each item's details by its code	O(1) per item

Result on set D: **685 rentals** sorted · **275 overdue before 2026-10-01, exactly the same rentals as a one-by-one scan** (the cell checks this with an `assert` and fails immediately if they differ).

Why step 1 must be right — Neo's binary search works only on a sorted list. It looks at the middle item and discards half, trusting that every item in that half is on the same side. If this part sorted wrongly, the halving would cut the wrong side and return the wrong rentals **with no error at all**. This is why `verify()` compares `merge_sort` with `sorted()` and checks that `sort_by_due_date` really sorts smallest first.

In the live web application — `services/inventory.py` calls `sort_by_due_date` in `open_borrows()` to sort the rentals not yet returned (the SQL does not sort them). The sorted list is then used in two places: Neo's `find_due_before` picks the rentals for the 08:00 LINE summary, and Neo's `count_overdue` counts the overdue rentals for the Workspace page. Step 3 (the hash table) is in the notebook, not in the web application.

3.9 Limitations

- **Merge sort does not adapt to the data** — data that is already sorted is still split and merged in full, O(n log n), unlike the Timsort in `sorted()`, which detects runs that are already sorted and skips that work.
- **It uses O(n) extra memory** — every level creates new lists by slicing and merging, unlike an in-place sort.
- **`key()` is called on every comparison** instead of being computed once and kept, as `sorted()` does. An expensive key makes it slower accordingly.
- **`due_date` must be in the form `YYYY-MM-DD`** — in another form, such as `DD/MM/YYYY`, comparing the text gives the wrong order with no error, and Neo's binary search would then be wrong as well.
- **`sort_by_due_date` returns only rentals with a `due_date`** — the result has m records, not n; the caller must know that items nobody has rented are not in it.

- **`group_by_status` silently skips unknown statuses** — if the data has a wrong status, the three groups add up to fewer than the total, with no warning.
- **Groups hold references, not copies** — changing a record in a group changes the original record too.
- **Not yet used in the live web application:** `bubble_sort` (for comparison only) · `sort_by_available` · `group_by_type` · `group_by_status` — these are used only in the notebook; the web application uses only `sort_by_due_date`.
- **There are no timing numbers for grouping or for `sort_by_due_date`** — the notebook times only sorting the whole set by `available` (Table M1).

3.10 Flowcharts

In `docs/data-course/flowcharts/`, as both draw.io and PDF files · the PDFs are also in the team folder: https://drive.google.com/drive/folders/1r2B5_h15vtCjMgca2rhnYVqcbX1wulWY

6. `flow_kul_6_merge_sort.pdf` — merge sort: check the base case → split in half and call itself on both sides → the `_merge` loop that compares the two heads with <= (the point that makes it stable) → append what is left, with a box showing the entry point the system really calls, `sort_by_due_date` (remove `None`, then hand the result to Neo's `find_due_before`).
7. `flow_kul_7_group.pdf` — the two kinds of grouping side by side: `group_by_type`, which creates a new key when it meets a new category, and `group_by_status`, which creates all 3 keys up front and skips unknown statuses. Both pass over the data once.

The English versions used in this report are `flow_kul_6_merge_sort_eng.pdf` and `flow_kul_7_group_eng.pdf`.

3.11 Related files

File	What it is
`backend-api/algorithms/sort_group.py`	All the code; standard library only · runs on its own with `python3 backend-api/algorithms/sort_group.py` (runs `verify()` first every time)
`backend-api/services/inventory.py`	Where the live web application calls `sort_by_due_date` (`open_borrows()`)
`docs/data-course/LEBIS_Final.ipynb`	The combined, already-run notebook of all three members — the numbers in this report come from this file (sections 4 and 5) · **Colab:** https://colab.research.google.com/drive/1K3JSlNVuN-mof98pjE09J_nFKhZEmyV8
`docs/data-course/flowcharts/flow_kul_6_merge_sort.pdf` · `flow_kul_7_group.pdf`	Both flowcharts (each with a matching `.drawio` file)
`scripts/build-notebooks.py`	Builds the notebooks from the real files in the repository

Conclusion — Three Algorithms Working Together

> *Written by the editor (Neo), summarising the three parts above. Every number comes from* `LEBIS_Final.ipynb`.

One question that needs all three members

Every morning the shop asks: **"Which rentals are overdue, and what is each item?"** No single algorithm can answer this alone. It takes three steps, each handing its result to the next.

Step	Who	What it does	Time
1	Kul — merge sort	Sorts every rental by due date	O(n log n)
2	Neo — binary search	Cuts out every rental due before today in one block	O(log n + k)
3	Kim — hash table	Looks up each item's details by its code	O(1) per item

Set D has **685 rentals**. These three steps find **275 rentals overdue before 2026-10-01**, **exactly the same rentals as a full one-by-one scan**.

Each step passes a plain list to the next, and nobody imports anyone else's code. If one member changed the format of the data they hand on, this notebook cell would fail immediately, so it serves both as an example of use and as a test that the three parts still fit together.

The live web application uses the same sequence. Every morning at 08:00 it sorts the rentals with merge sort, then uses binary search to find what is due, and sends the shop owner a summary on LINE.

Key numbers of the report

Each hand-written algorithm against the simplest method that does the same job, on set D (1,000 records).

Part	Task	Simple method	Hand-written	Speed-up	Python's own (baseline)
1 — Kim	Look up every code once	List scan 21.753 ms	Hash table 1.530 ms	**14.2×**	`dict` 0.053 ms
2 — Neo	Look up every code once	One-by-one scan 21.812 ms	Binary search 1.759 ms	**12.4×**	Scanning the whole list (the "simple method" column)
3 — Kul	Sort the whole set once	Bubble sort 90.516 ms	Merge sort 2.619 ms	**34.6×**	`sorted()` 0.124 ms

Complexity of every algorithm in the report

Part	Algorithm	Time (average)	Time (worst)	Memory
1	Lookup · insert · delete with the hash table	O(1)	O(n)	O(n)
2	Binary search	O(log n)	O(log n)	O(1)
2	Range query `find_due_before`	O(log n + k)	O(log n + k)	O(k)
2	Counting (`count_*`)	O(n)	O(n)	O(1)
3	Merge sort	O(n log n)	O(n log n)	O(n)
3	Grouping (`group_by_*`)	O(n)	O(n)	O(n)

The reasoning behind each value is in the Big-O section of that part.

What all three parts found

1. On small data, the simple method wins. All three parts saw the same thing on sets A–C (10–12 records): a list scan beat the hash table, a one-by-one scan beat binary search, and bubble sort matched or beat merge sort. When n is small, the fixed cost of the clever part (hashing, finding the middle, splitting) outweighs the real work. Big-O tells us what **grows more slowly**, not what is **faster at every n**. Measuring only the small sets would lead to the wrong conclusion in all three parts.

2. On large data, the Big-O difference shows up in the measurements. From set A to set D the data grows 100 times. Part 2 measured binary search growing 293 times, close to the 300 times O(log n) predicts, and the hash table growing 99 times, close to the predicted 100.

3. Python's built-ins beat every hand-written version — because of C, not because of the algorithm. `dict`, `sorted()` and `bisect` are written in C, so they are many times faster than the same idea written in Python. They are in the same Big-O class, however: they grow at the same rate as the data grows, and differ only by a constant factor.

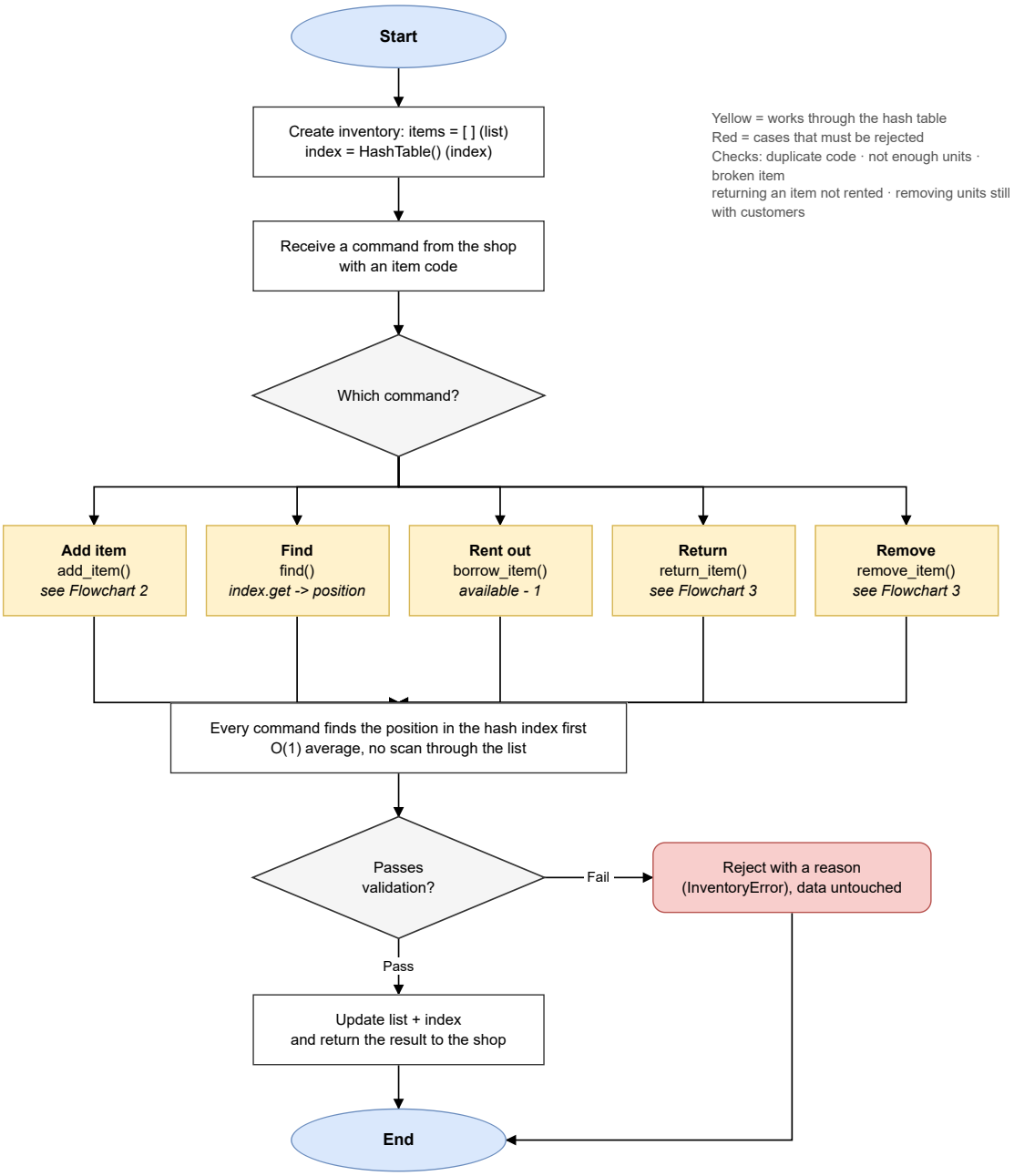
4. No single data structure answers every question. A hash table is fastest when the exact code is known, but it cannot answer range questions and cannot be sorted. A sorted list answers range questions, but it has to be sorted first, which is merge sort's job. The real system therefore keeps both, and chooses by the kind of question being asked.

Appendix — Flowcharts

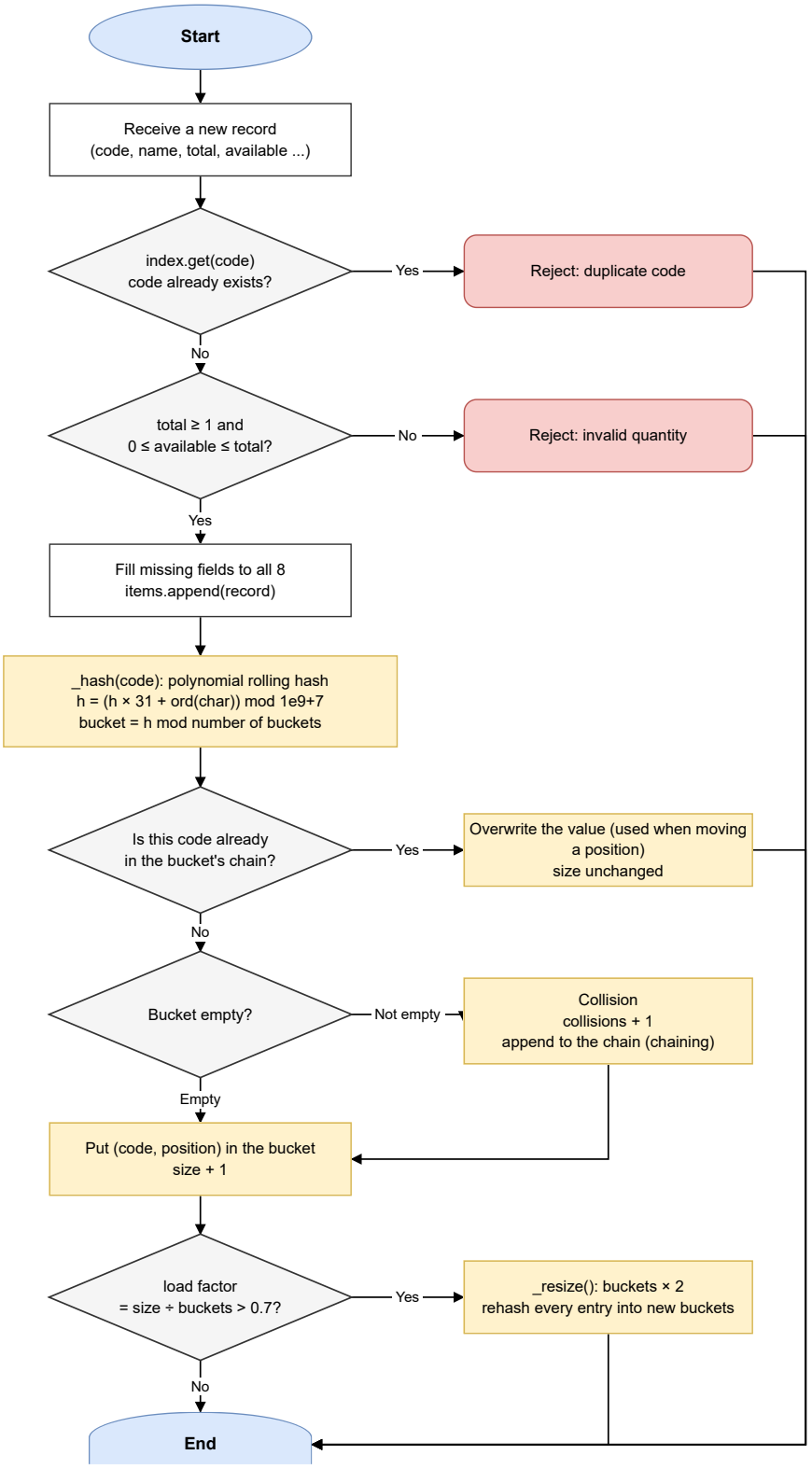
All flowcharts in the report, in figure order, one per page. The draw.io sources and PDFs are in `docs/data-course/flowcharts/`.

Figure	Subject	Author
1	System overview: inventory (list + hash index)	Kim
2	Adding an item (add_item + HashTable.insert)	Kim
3	Removing an item (remove_item) and returning (return_item)	Kim
4	Search: three paths (hash · binary · linear)	Neo
5	Counting: count_available · count_by_status · count_overdue	Neo
6	Sorting: merge_sort (split in half → merge)	Kul
7	Grouping: group_by_type and group_by_status	Kul

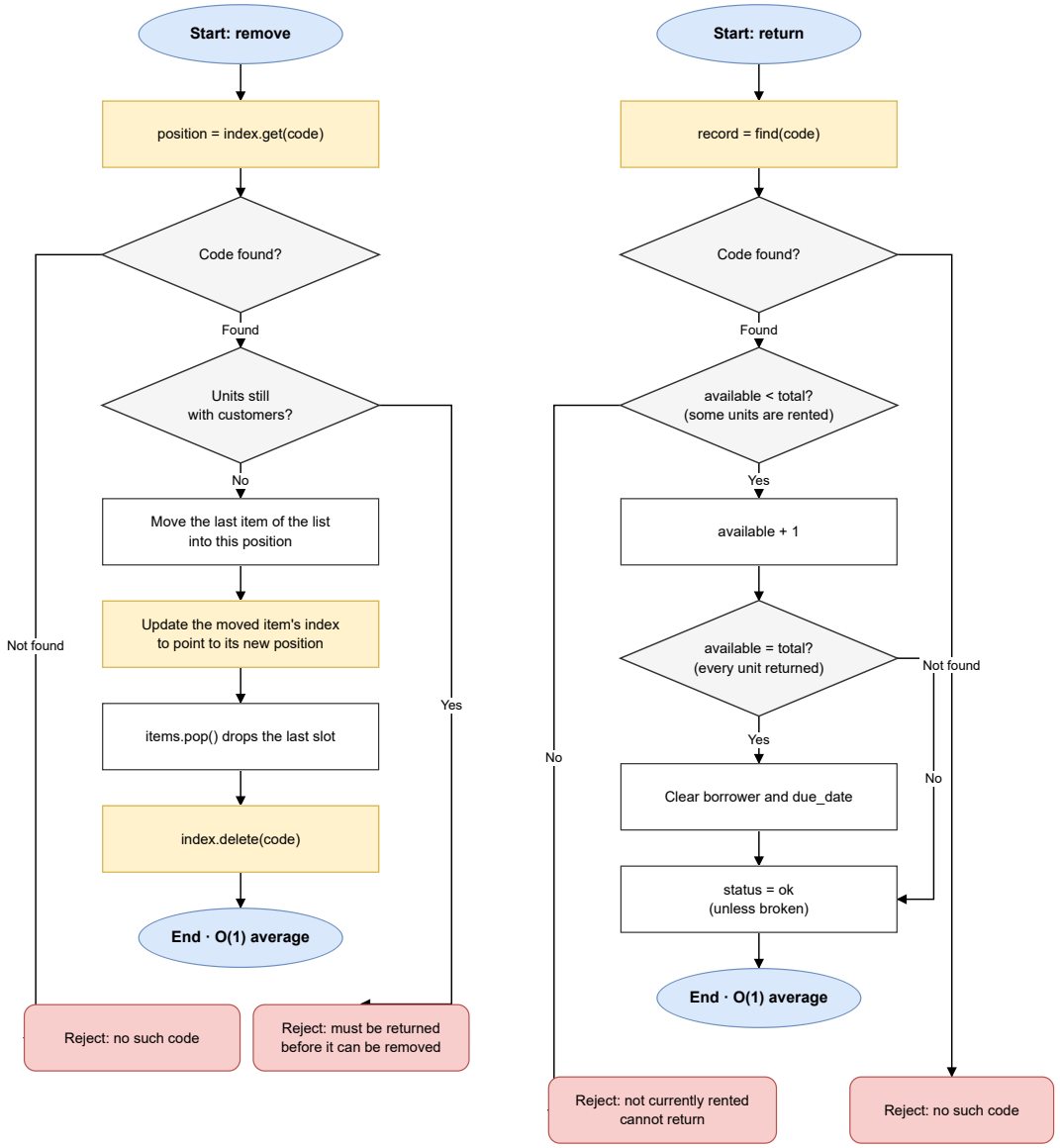
Flowchart 1 — System overview: inventory (list + hash index)



Flowchart 2 — Adding an item (add_item + HashTable.insert)



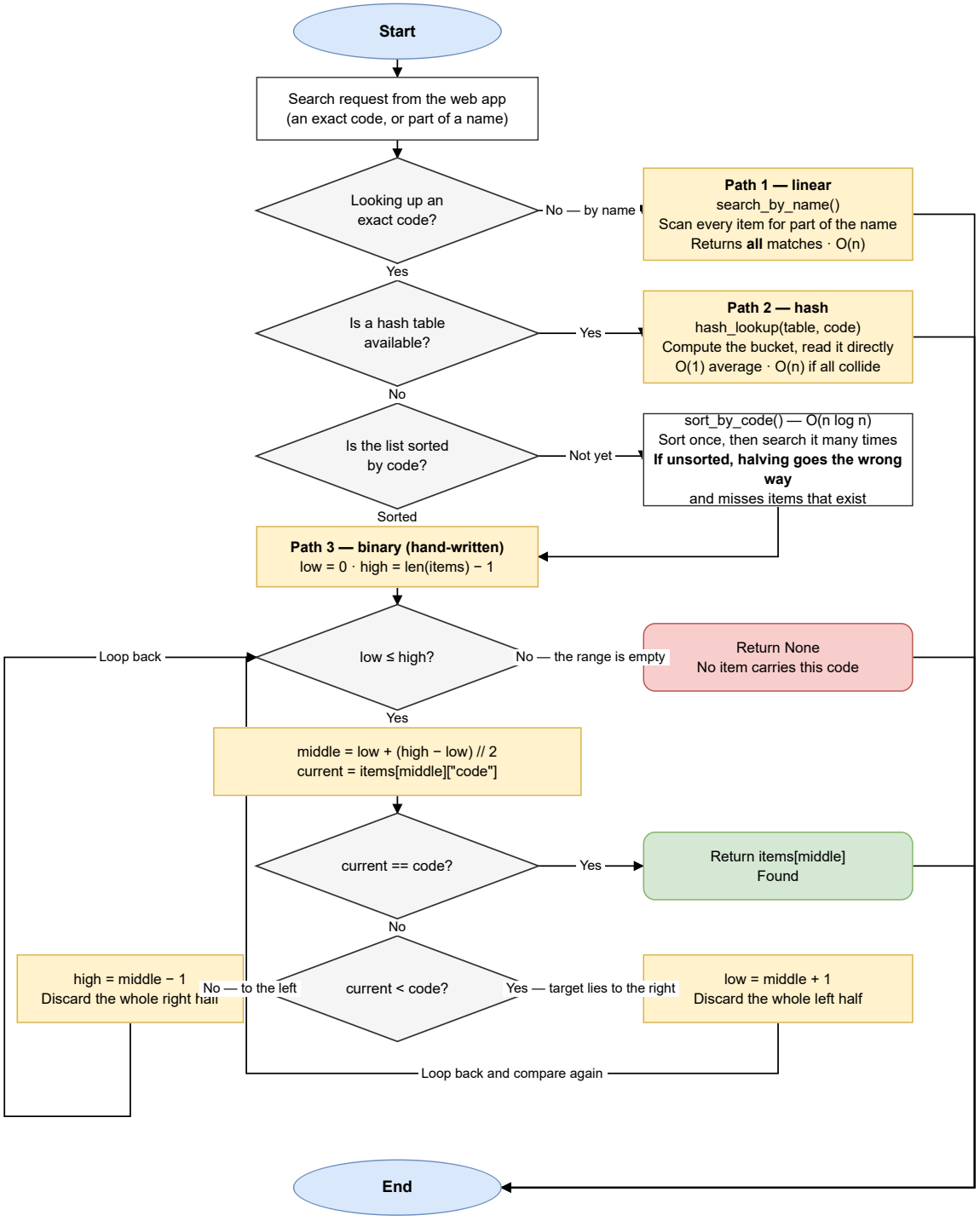
Flowchart 3 — Removing an item (remove_item) and returning (return_item)



Why not items.pop(position) directly?
Deleting mid-list shifts every later item = O(n)
and their positions in the index all become wrong
swap-and-pop updates only one index entry = O(1)

Flowchart 4 — Search: three paths (hash · binary · linear)

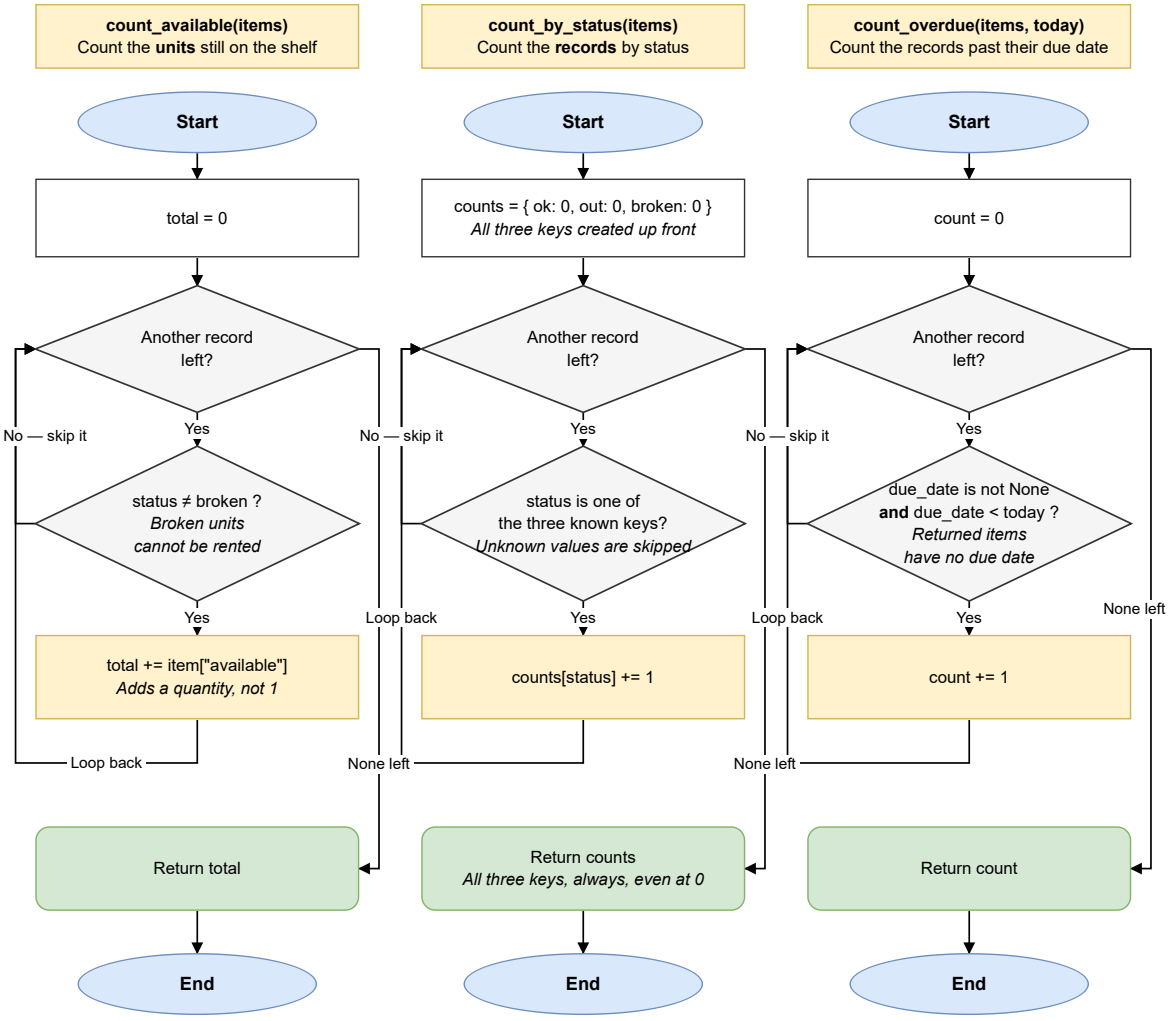
search.py · The path is chosen by the kind of question asked; binary_search, the hand-written one, is drawn in full



Why one round halves the work Each comparison halves what is left to examine, so 1,000 items need about 10 comparisons ($2^{10} = 1{,}024$), against roughly 500 on average for a linear scan · Written as a loop rather than recursion, so it adds no stack frames: memory is O(1), not O(log n)

Flowchart 5 — Counting: count_available · count_by_status · count_overdue

search.py · All three are a single pass over the data — the same skeleton, differing only in the condition and in what is added

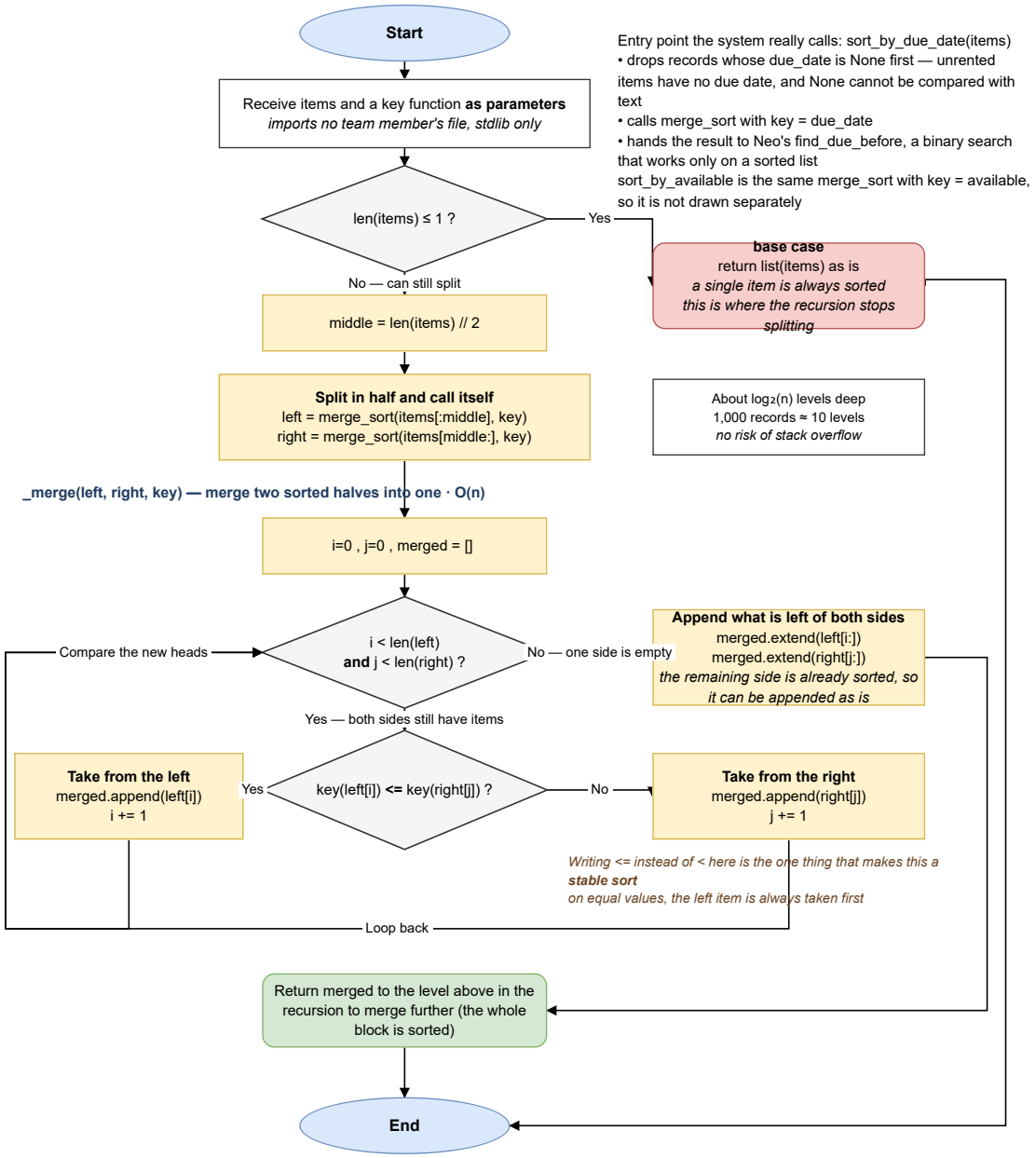


Why all three are O(n), with no shortcut

- Counting differs from searching: there is **no "right one" to stop at**. Every item must be examined before the number is known, so none can be skipped.
- Binary search does not apply — halving needs one target that can be compared as greater or smaller, and counting has no such target.
- A hash table does not help either: it answers "where is this one", never "how many satisfy the condition".
- **Memory is O(1) for all three** — count_by_status holds three fixed slots, which do not grow with n.
- count_by_status produces all three numbers in a **single pass** rather than three separate loops, reading the data three times less.
- Dates compare correctly as plain strings because YYYY-MM-DD runs from the largest unit to the smallest; stored as DD/MM/YYYY this stops being true at once.

Flowchart 6 — Sorting: merge_sort (split in half → merge)

sort_group.py · built as recursion that splits in half until one item is left, unlike hash_table.py, which computes the position straight from the key, and unlike count_* in search.py, which walks the data once



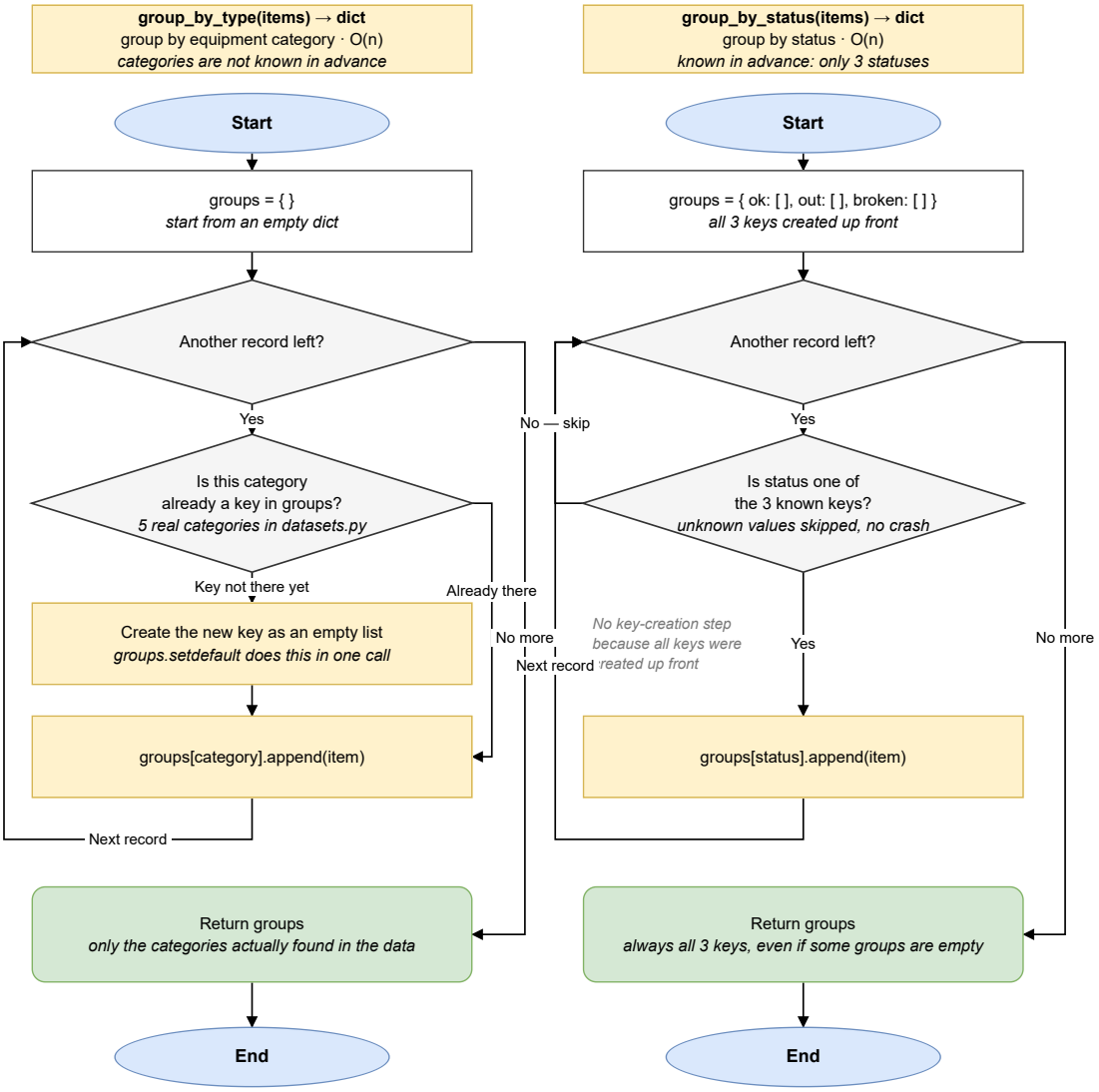
Entry point the system really calls: sort_by_due_date(items)
• drops records whose due_date is None first — unrented items have no due date, and None cannot be compared with text
• calls merge_sort with key = due_date
• hands the result to Neo's find_due_before, a binary search that works only on a sorted list
sort_by_available is the same merge_sort with key = available, so it is not drawn separately

Why splitting in half and merging is faster than swapping pair by pair

• Splitting in half until one item is left gives **log₂(n) levels**, and merging each level costs **O(n)**, for **O(n log n)** in total
• Bubble sort compares every pair over and over, so it is **O(n²)** — measured on set D with 1,000 records, **merge sort is 34.6 times faster than bubble sort**
• Writing **<= instead of <** when comparing the left side makes it a **stable sort**, so records with the same due_date keep their original order instead of being swapped for no reason
• It **always returns a new list and never changes the original**, because someone else may hold the same list, just like sort_by_code in search.py
• The sorted list from this file is what **find_due_before (Neo's binary search)** uses directly; if it were sorted wrongly, the halving would cut the wrong side **with no error at all**

Flowchart 7 — Grouping: group_by_type and group_by_status

sort_group.py · receives items as a parameter, imports no team member's file · walks the data once like count_by_status, but stores whole records in buckets instead of only counting



Why one pass into buckets is faster than searching category by category

- The bucket method reads the data **once**, however many categories there are, so it is **O(n)**
- Searching category by category (first scan for cameras, then scan again for lenses) reads the whole data again **once per category**, so it is **O(n × number of categories)**
- The team's data set has **5 categories** (camera · lens · tripod · light · microphone), so the category-by-category method must read the data **5 times** to get the same result
- **Memory is O(n)** for both functions, because every record is stored in some bucket and none is discarded, unlike counting, which uses **O(1)**
- Unlike **Neo's count_by_status**, which keeps only **counts**, this keeps **whole records**, for when you need to know "what is there", not just "how many"
- Both functions **always return the keys they promise** — group_by_status returns all 3 keys even if some groups are empty, so callers do not have to write .get() guards everywhere, which someone would certainly forget somewhere